



Machine Learning

Predicting Customer Churn for a Telecommunications Company

Project Report

Supervisor:

Prof Giacomo Fiumara

Student:

Aditi Dole(533674)

Introduction

Machine learning aims to perform tasks and learn from data using algorithms. It is a field of artificial intelligence and computer science. The algorithms allow us to identify patterns and relationships in data, replicate human learning processes and improve accuracy over time.

The project's objective is to use a synthetic dataset to predict customer churn, analyse factors influencing churn and explore customer segmentation.

Understanding the dataset

- a) `Data.Info()`: The info function gives information about the column names and their data type along with non-null value count. The main datatypes include integer, float and object. The shape of the dataset is (3749,17). The dataset describes the personal characteristics of the customer, Services given and charges for the same.
1. CustomerID: A unique number for each customer consisting of alphabets and numbers.
 2. Age: of each customer.
 3. Gender : Defines if the customer is male or female
 4. Tenure: Number of months the customer is staying
 5. Service_Internet: Which type of medium is used to provide internet.
 6. Service_Phone and Tv : Service for phone and tv opted or not
 7. Contract: Kind of contract. 3 categories
 8. Payment Method : Payment method used. 4 Categories
 9. Monthly charges: Charges per month.
 10. Total Charges: Total Charges till date.
 11. Streaming media: Service opted or not.
 12. Security and technical support: Provision provided or not.
 13. Churn: Is the customer retained or not.

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3749 entries, 0 to 3748
Data columns (total 17 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   Unnamed: 0          3749 non-null  int64  
1   CustomerID          3749 non-null  object  
2   Age                 3562 non-null  float64 
3   Gender              3749 non-null  object  
4   Tenure              3749 non-null  int64  
5   Service_Internet    3028 non-null  object  
6   Service_Phone       3749 non-null  object  
7   Service_TV          3749 non-null  object  
8   Contract            3749 non-null  object  
9   PaymentMethod       3562 non-null  object  
10  MonthlyCharges      3749 non-null  float64 
11  TotalCharges        3749 non-null  float64 
12  StreamingMovies     3749 non-null  object  
13  StreamingMusic       3749 non-null  object  
14  OnlineSecurity      3749 non-null  object  
15  TechSupport         3749 non-null  object  
16  Churn               3749 non-null  object  
dtypes: float64(3), int64(2), object(12)
memory usage: 498.0+ KB

```

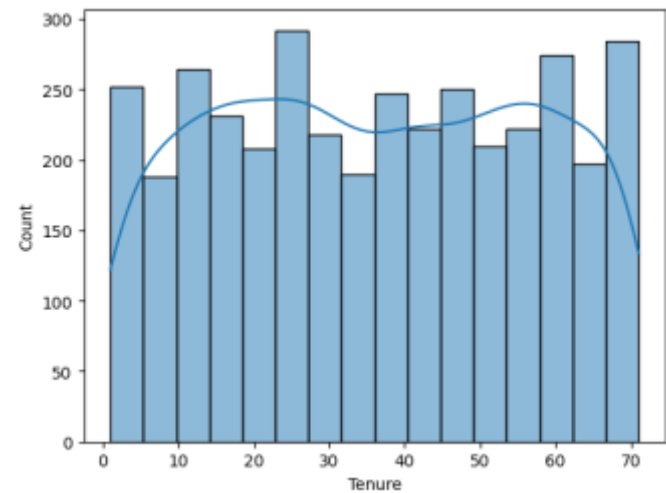
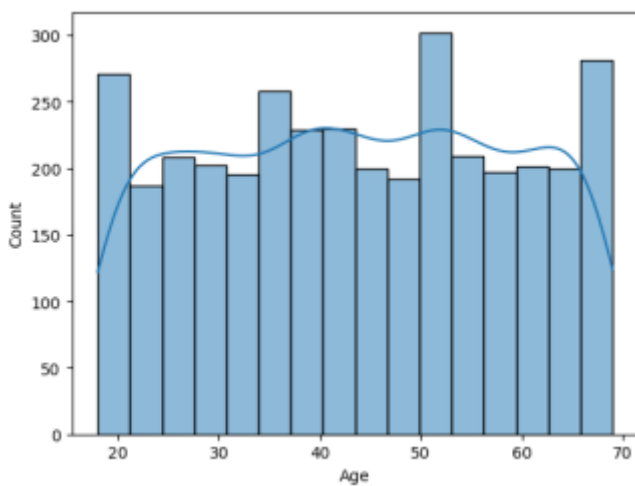
- b) `Data.describe()`: The describe function describes the data statistically. We get information such as the total count, mean and the standard deviation. The minimum values and maximum values are also given. Additionally, the quartiles are provided.

	Unnamed: 0	Age	Tenure	MonthlyCharges	TotalCharges
count	3749.000000	3562.000000	3749.000000	3749.000000	3749.000000
mean	1874.000000	43.655531	36.264070	75.844318	2718.968266
std	1082.387408	14.914474	20.505528	73.062971	3211.879149
min	0.000000	18.000000	1.000000	20.000000	13.190000
25%	937.000000	31.000000	19.000000	44.570000	1076.240000
50%	1874.000000	44.000000	36.000000	69.590000	2132.260000
75%	2811.000000	56.000000	54.000000	95.540000	3619.710000
max	3748.000000	69.000000	71.000000	1179.300000	79951.800000

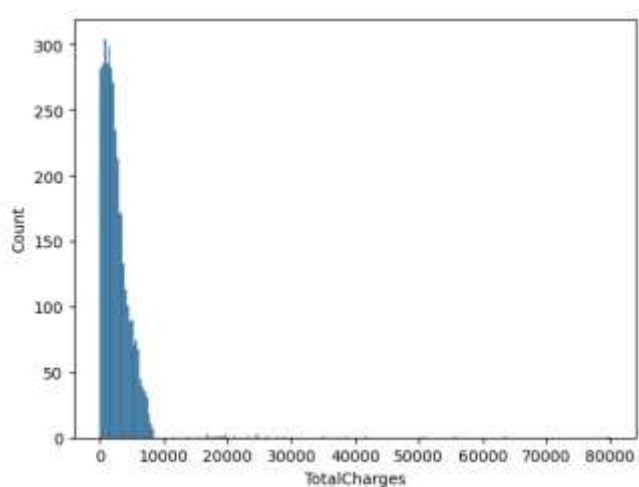
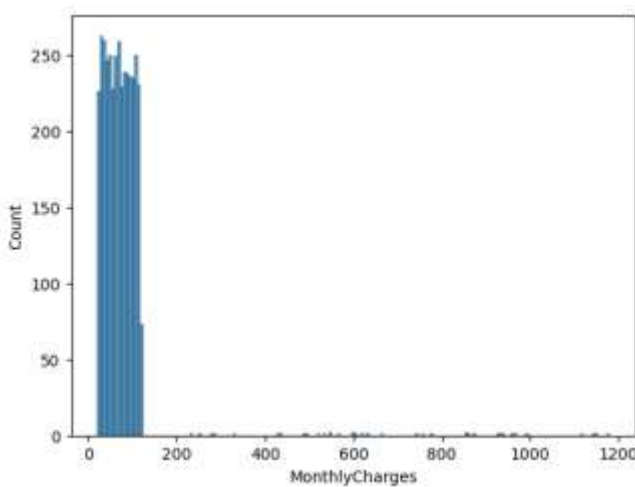
- c) Data visualisation: The different plots allow for observing the data distribution and finding any anomalies. The numerical features are visualised using boxplots and histograms.

```
for col in numerical_features:
    plt.figure(figsize=(10, 6))
    sns.histplot(data[col], kde=True)
    plt.title(f"Distribution of {col}")
    plt.figure(figsize=(10, 6))
    sns.boxplot(data[col])
    plt.title(f"Boxplot of {col}")
    plt.figure(figsize=(10, 6))
    sns.scatterplot(data[col])
    plt.title(f"Scatterplot of {col}")
```

a. Histogram:



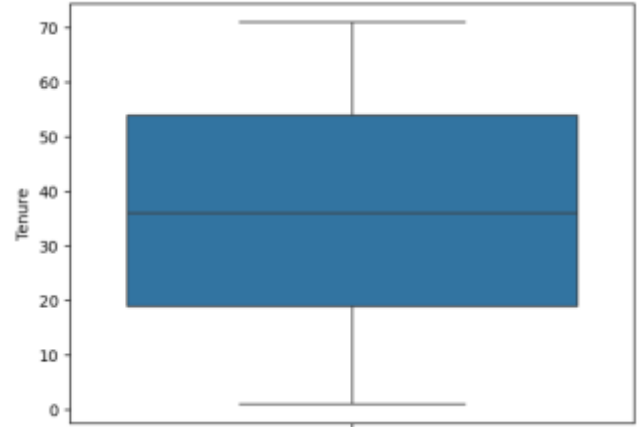
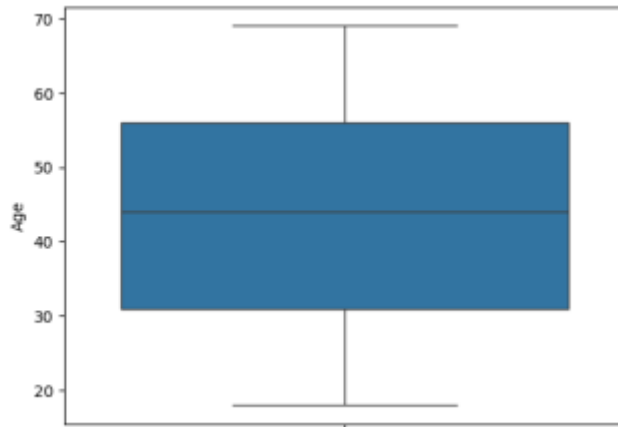
Age and tenure seem to be distributed normally.



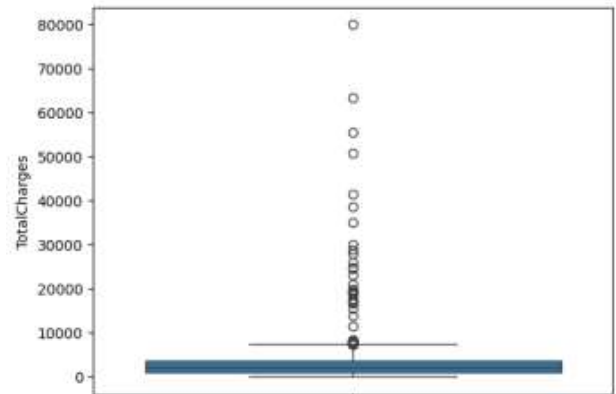
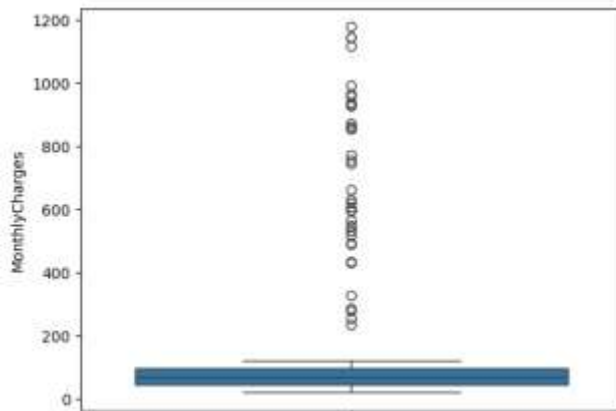
The charges are appearing skewed in the left direction. This tells that the presence of outliers is evident.–

- b. Boxplot: In descriptive statistics, a box plot is a method for demonstrating graphically the locality, spread and skewness groups of numerical data through their quartiles.

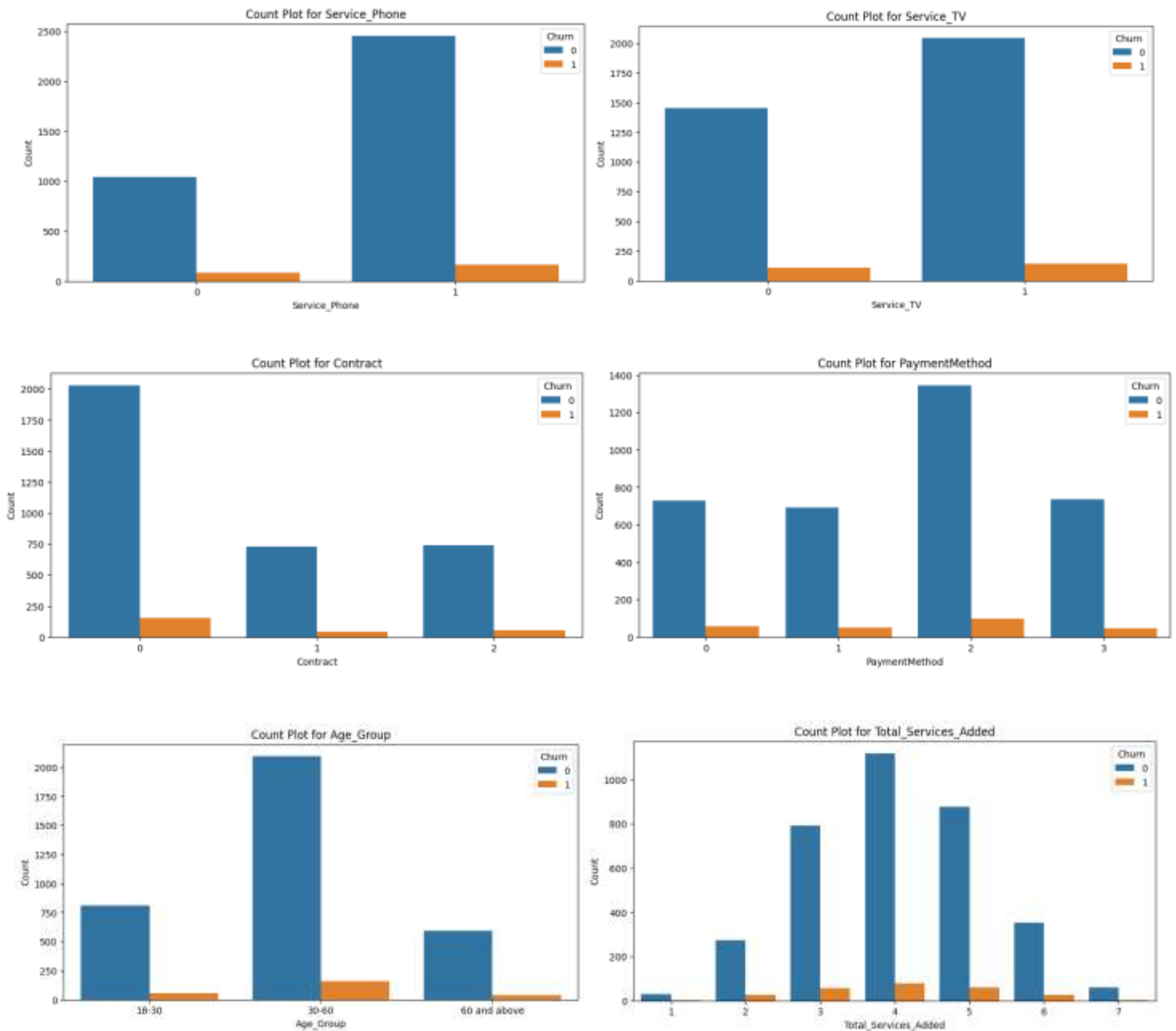
The Boxplot for age and tenure are quite similar.



The boxplot for monthly charges and total charges show outliers.



- c. Count Plots: The below count plots show distribution of categorical features concerning Churn. This shows how each feature affects the target variable. These count plots were visualised after encoding and also includes the new features that were crafted.



Data Pre-processing and EDA:

- a) Dropping Columns : In order to reduce the complexity of the data, unnecessary columns were removed. Customer id contains no useful information. As well as Gender does not contribute much.

```
#drop columns
data.drop(columns=['CustomerID','Unnamed: 0','Gender'],inplace=True)
✓ 0.0s
```

- b) Missing values: There are features that contain null values. Age: 187, Service_Internet: 721 and Payment Method:187 respectively. There are different approaches to handle missing values. I have used fillna() method for 'Age'. This imputes the mean inplace of missing values. I created a function to impute random choice in the 'PaymentMethod' and 'Service_Internet'.

```
data.isnull().sum()
Age      187
Tenure    0
Service_Internet  721
Service_Phone    0
Service_TV       0
Contract         0
PaymentMethod    187
MonthlyCharges   0
TotalCharges     0
StreamingMovies  0
StreamingMusic   0
OnlineSecurity   0
TechSupport      0
Churn            0
dtype: int64
```

```
def fill_na_random(data, str1, str2,str3,str4):

    return data.apply(lambda x:
np.random.choice([str1, str2,str3,str4]) if pd.isna(x)
else x)
```

- c) Treating outliers: With the help of boxplots; I could see there are outliers in the monthly charges and total charges. It is essential to treat them else they may affect the model performance ahead.

I have used the Interquartile Range to detect the outliers. Both features are treated with same strategy. I created thresholds (values 1.5 times the quartiles) using the first and third quartile. The values crossing the thresholds are replaced with the respective threshold value. Following is the comparison of Total Charges before and after treating the outliers.

```
def treat_outliers(data, col):
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1

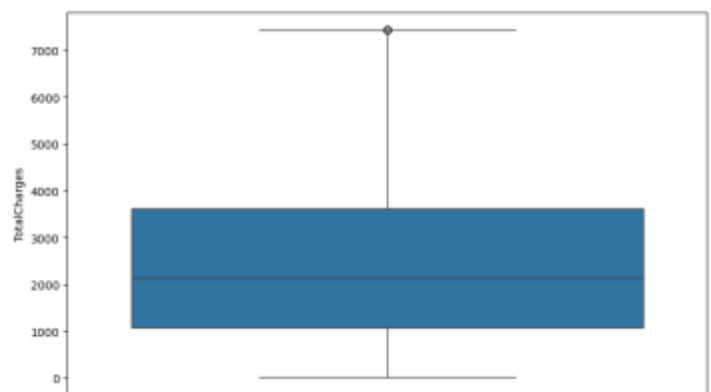
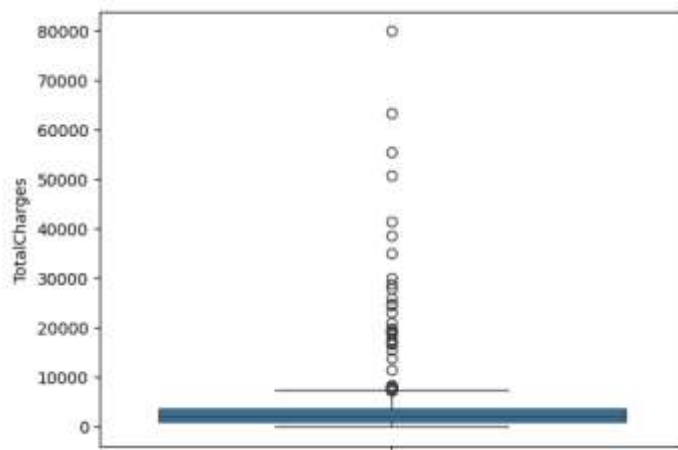
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)]

    # Display the list of outliers
    outliers_list = outliers[col].tolist()
    print(f'outliers_list_{col}', outliers_list)

    # Capping outliers
    data[col] = np.where(data[col] > upper_bound, upper_bound,
                        np.where(data[col] < lower_bound, lower_bound, data[col]))
    data[col] = np.ceil(data[col])

    return outliers_list

print(treat_outliers(data, 'MonthlyCharges'))
print(treat_outliers(data, 'TotalCharges'))
```



d) Encoding: There are many categorical values in the dataset. Some contain specific categories and others have boolean values. It is necessary to bring all these values in one format. I have used label encoder to encode all features.

Feature	Values Encoded
All services	Yes = 1 No = 0
Service_Internet	Fiber optic = 1 DSL = 0
Payment_Method	Bank transfer=0 Credit card=1 Electronic check=2 Mailed check = 3

Contract	Month to Month = 0 One year = 1 TwoYear= 2
-----------------	--

```
#run once
le = LabelEncoder()

le.fit(['Yes', 'No'])
categorical_columns = ['Service_Phone', 'Service_TV',
'StreamingMovies', 'StreamingMusic', 'OnlineSecurity',
'TechSupport', 'Churn']
for col in categorical_columns:
    data[col] = le.transform(data[col])

DSL, 'Fiber optic'])
vice_Internet'] = le.transform(data['Service_Internet']) # if DSL =0, else 1

# Label encoding contract and payment method
le.fit(['Bank transfer', 'Credit card', 'Electronic check', 'Mailed check']) #0,1,
data['PaymentMethod'] = le.transform(data['PaymentMethod'])
le.fit(['Month-to-month', 'One year', 'Two year'])
data['Contract'] = le.transform(data['Contract'])
```

e) CHI-2 test: The test is used to find associations among categorical features. Here I tried to find an association between Churn and other categorical values. We find that there is no association whatsoever.

```
categorical_columns = ['Service_Internet', 'Service_Phone', 'Service_TV', 'C
'PaymentMethod', 'StreamingMovies', 'StreamingMusic', 'OnlineSecurity', 'Tec

# Function to perform Chi-Square test
def chi_square_test(column):
    y = data['Churn']
    contingency_table = pd.crosstab(y, data[column])
    chi2, p, dof, expected = chi2_contingency(contingency_table)
    print(f"Chi-Square Test for {column}")
    print("Chi-Square Statistic:", chi2)
    print("P-value:", p)
    print("Degrees of Freedom:", dof)
    print("Expected Frequencies:\n", expected)
    if p < 0.05:
        print(f"Reject the null hypothesis: There is an association between")
    else:
        print(f"Fail to reject the null hypothesis: There is no association")

# Perform Chi-Square test for each categorical column
for column in categorical_columns:
    chi_square_test(column)
```

f) Correlation: This is the correlation of the features with respect to Churn. Most features are negatively correlated. Monthly Charges is the most positively correlated feature.

```

Correlation with target variable:
Churn          1.000000
MonthlyCharges 0.216844
TechSupport    0.018568
Service_Internet -0.003281
Age            -0.003401
StreamingMovies -0.004435
Service_TV     0.005389
PaymentMethod  -0.006545
StreamingMusic -0.009525
Contract       -0.011690
OnlineSecurity 0.012379
Service_Phone  -0.019730
TotalCharges   -0.276758
Tenure         -0.392958
Name: Churn, dtype: float64

```

Feature Engineering:

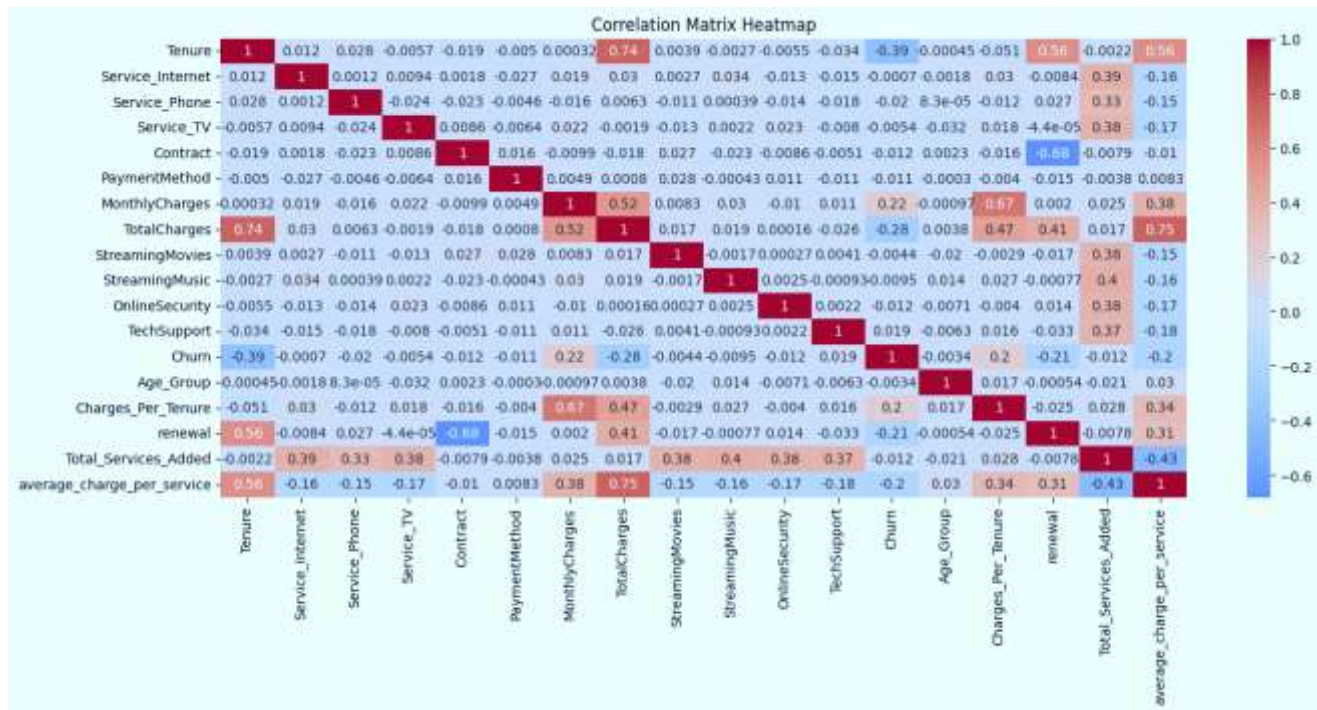
- a) New Features: I created new features based mainly on Total Charges, Tenure and Services.
- Charges per Tenure = Total charges/ tenure
 - Renewal of contract : If the contract is month to month or year wise ; how many times it has been renewed.
 - Total Services Added = Sum of the services columns for each row.
 - The average charge per service = total charges/ total services added
 - Age group: 18-30, 30-60, 60 and above

	Age_Group	Charges_Per_Tenure	renewal	Total_Services_Added	average_charge_per_service
0	1	71.653077	1.0	3	310.496667
1	2	111.420000	0.0	4	362.115000
2	1	116.628833	60.0	4	1749.432500
3	1	78.107544	57.0	5	890.426000
4	1	30.187115	2.0	6	261.621667

b) Correlation:

Correlation tells how dependent the features are on each other. A positive correlation means there is a direct relationship. And negative means an indirect relationship. We have to observe the correlation with Churn. We see the following relations:

- **Positive correlation w.r.t Churn:** Monthly charges, Charges per Tenure
- **Negative correlation w.r.t Churn:** Tenure, Total Charges, renewal and average charge per service
- Otherwise there is quite less correlation among the features.



c) Feature Scaling: It is important to scale features before selecting. There are two main approaches Standard Scaler and Min Max Scaler. Here using Min max scaler is more useful as it scales the data in the range of 0 to 1.

Service_TV	Contract	PaymentMethod	MonthlyCharges	TotalCharges
0.0	0.5	1.000000	0.342105	0.123703
1.0	1.0	1.000000	0.598684	0.193370
1.0	0.0	1.000000	0.638158	0.941113
1.0	0.0	0.000000	0.388158	0.598167
1.0	1.0	0.666667	0.072368	0.209675

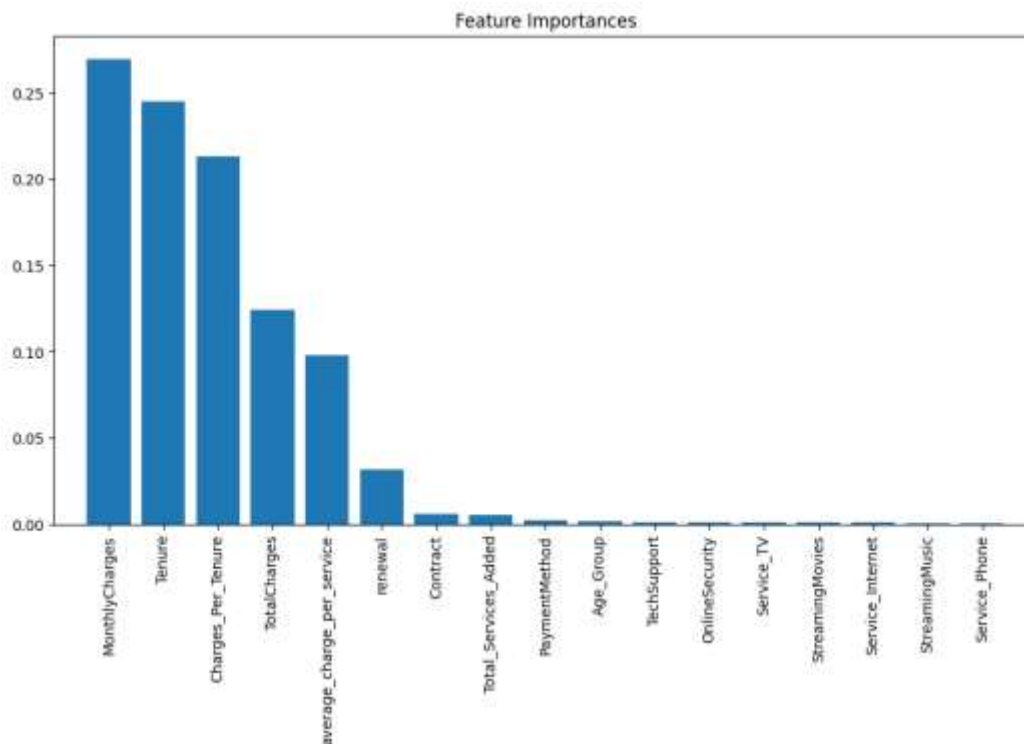
d) Feature Selection: Feature selection allows to train data with most important features. There are various techniques such as recursive feature elimination, using decision trees or random forest classifiers, or sequential feature selector.

I selected the Random classifier to get feature ranking according to importance. I have selected the top 9 features. This keeps some of the original features and also additional ones.

```

Feature ranking:
1. MonthlyCharges (0.253359)
2. Tenure (0.249571)
3. Charges_Per_Tenure (0.205718)
4. TotalCharges (0.146037)
5. average_charge_per_service (0.095563)
6. renewal (0.036089)
7. Total_Services_Added (0.003767)
8. Contract (0.002942)
9. PaymentMethod (0.001699)
10. Age_Group (0.000924)
11. Service_TV (0.000716)
12. TechSupport (0.000709)
13. Service_Phone (0.000689)
14. StreamingMovies (0.000618)
15. Service_Internet (0.000571)
16. StreamingMusic (0.000565)
17. OnlineSecurity (0.000462)

```



Model training:

After creating a new data frame with selected features, I split the data into training and test sets with a ratio 80:20. The training set consists 2999 instances and test set 750. To evaluate the performance of the model the metrics are given as follows:

- Accuracy: is the ratio of correctly predicted observations to the total observations.
It is given by $\text{True Positives} + \text{True Negatives} / \text{Total observations}$

- b. Cross Validation Accuracy: is the average accuracy obtained by training the model on different subsets of the data and validating it on complementary subsets.
- c. Precision: is the ratio of correctly predicted positive observations to the total predicted positives. It is given by $\text{True Positives} / (\text{True Positives} + \text{False Positives})$.
- d. Recall: is the ratio of correctly predicted positive observations to the all observations in actual class. It is given by $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$.
- e. F1 score: is the harmonic mean of precision and recall, providing a single metric that balances both concerns.

In my opinion running the model on both training and test set gives an insight on fitting of the data. Hence I have included both results here. The confusion matrix returns the actual count of values for the metrics.

```
classifiers = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=3),
    'Poly_SVC': SVC(kernel="poly", degree=3, coef0=1, C=5),
    'sgd': SGDClassifier(),
    'rf': RandomForestClassifier(),
}
```

```
for clf_name, clf in classifiers.items():
    pipeline = Pipeline([
        ('classifier', clf)
    ])
    # Perform cross-validation on the training set
    scores = cross_val_score(pipeline, X_train, y_train, cv=5)
    print(f'{clf_name} Cross-Validation Accuracy: {scores.mean()}')

    # Fitting the classifier to the training data
    pipeline.fit(X_train, y_train)

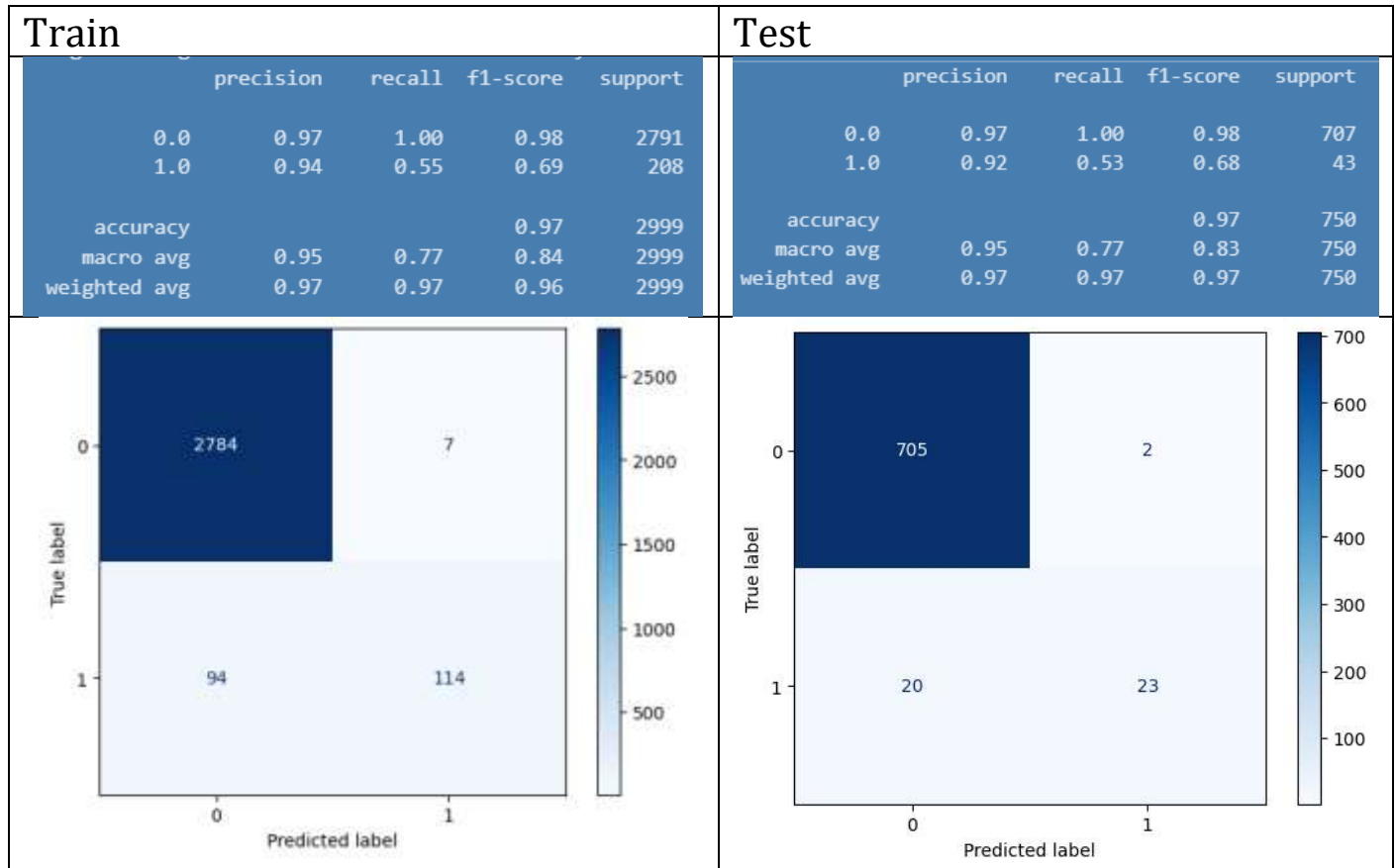
    # Predict on train and test set
    y_train_pred = pipeline.predict(X_train)
    y_pred = pipeline.predict(X_test)
```

```
# Performance and Confusion Matrix
print(classification_report(y_train, y_train_pred))
cm_train = confusion_matrix(y_train, y_train_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_train)
disp.plot(cmap=plt.cm.Blues)
plt.title(f'{clf_name} Train')
plt.show()

print(classification_report(y_test, y_pred))
cm_test = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_test)
disp.plot(cmap=plt.cm.Blues)
plt.title(f'{clf_name} Test')
plt.show()
```


a) Logistic Regression:

Logistic Regression is used to estimate the probability that an instance belongs to a particular class. If the estimated probability is greater than 50%, then the model predicts that the instance belongs to that class (called the positive class, labelled “1”), or else it predicts that it does not (i.e., it belongs to the negative class, labelled “0”). This makes it a binary classifier. ~Hands-On Machine Learning text book



Observations:

The model performance is consistent along the train and test set. The recall is not very good for the Churn == Yes. There is a 45% of false negatives. The Cross-Validation Accuracy is 96%.

b) K-Nearest Neighbors: It works on the principle that when trying to classify a new data point, the algorithm looks at the 'k' closest data points in the training set. The new data point is then assigned the most common class among its 'k' nearest neighbors. Here K is 3.

Train					Test				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0.0	0.99	1.00	1.00	2791	0.0	0.98	0.99	0.99	707
1.0	0.95	0.93	0.94	208	1.0	0.79	0.72	0.76	43
accuracy			0.99	2999	accuracy			0.97	750
macro avg	0.97	0.96	0.97	2999	macro avg	0.89	0.85	0.87	750
weighted avg	0.99	0.99	0.99	2999	weighted avg	0.97	0.97	0.97	750

KNN Train

	Predicted 0	Predicted 1
True 0	2781	10
True 1	14	194

KNN Test

	Predicted 0	Predicted 1
True 0	699	8
True 1	12	31

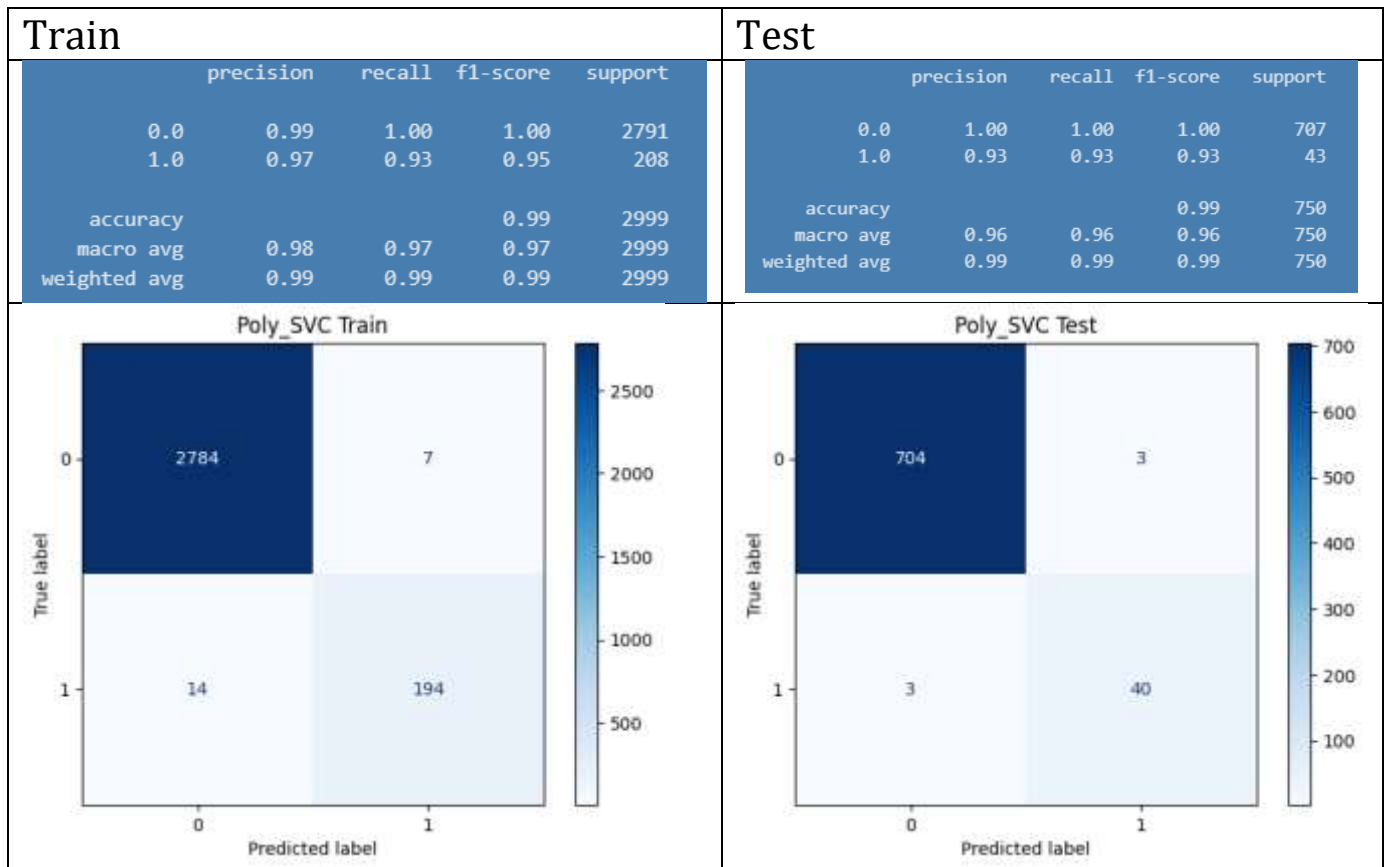
Observations:

The model works better on the training set than the test set. This suggests overfitting of the data. There is 97% accuracy of the cross-validation.

c) Support Vector Machine:

SVM uses an algorithm to find an optimal hyperplane between the data points for classification. However, using the poly kernel allows the creation of complex decision boundaries by transforming the data into a higher-dimensional space using polynomial functions.

The classifier uses the polynomial kernel, uses a degree of 3, coef0= 1, and C=5. C is the regularization parameter and uses L2 penalty as default.



Poly_SVC Cross-Validation Accuracy: 0.9879938786867

Observations:

The model performs consistently on the train and test set. The Cross-validation accuracy is also 98%. The metrics on the train set are slightly better performing than the test set. The overall Accuracy is 99% which can be said is well well-performing.

Tuning models:

Hyper tuning the models requires a pre-defined grid of parameter for the classifier. It then traverses the different values of attributes and returns the best estimator and model based on accuracy. Here the grid search uses cross validation 5 times.

```
classifiers = {'sgd':SGDClassifier(),
               'rf':RandomForestClassifier(),}
param_grid = {'rf': {'clf__n_estimators': [50, 100, 200],
                    'clf__max_depth': [None, 10, 20],
                    'clf__min_samples_split': [2, 5, 10],
                    'clf__min_samples_leaf': [1, 2, 4]
               },
              'sgd':{
                    'clf__loss': ['hinge', 'log_loss', 'modified_huber'],
                    'clf__penalty': ['l2', 'l1', 'elasticnet'],
                    'clf__alpha': [0.0001, 0.001, 0.01],
                    'clf__learning_rate': ['constant', 'optimal', 'invscaling'],
                    'clf__eta0': [0.01, 0.1, 1],
                    'clf__max_iter': [1000, 2000, 3000]
                }
              }
# hyperparameter grid
for clf_name, clf in classifiers.items():
    print(f"Tuning {clf_name}...")
    pipeline = Pipeline([
        ('clf', clf)
    ])

    grid_search = GridSearchCV(pipeline, param_grid[clf_name], cv=5, n_jobs=-1, scoring='accuracy')

    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_
```

```
#evaluate on training
y_train_pred = best_model.predict(X_train)
print("Best parameters found:", grid_search.best_params_)
print("Best cross-validation accuracy:", grid_search.best_score_)
print("Train set performance:")
print(classification_report(y_train, y_train_pred))
cm = confusion_matrix(y_train,y_train_pred)

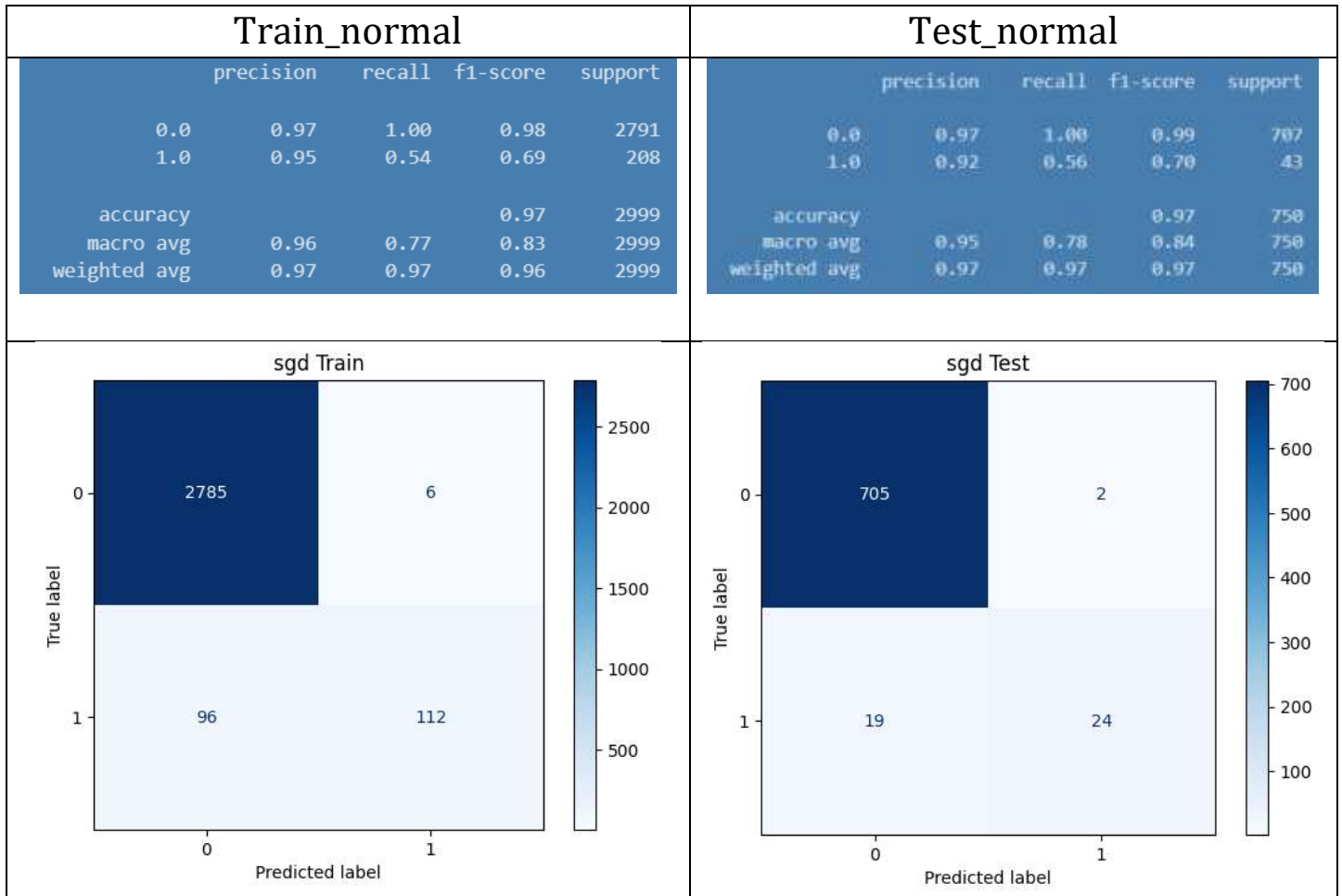
# Display the confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap=plt.cm.Blues)
plt.show()

# Make predictions on test
y_pred = best_model.predict(x_test)
print("Best parameters found:", grid_search.best_params_)
print("Best cross-validation accuracy:", grid_search.best_score_)
print("Test set performance:")
print(classification_report(y_test, y_pred))
cm2 = confusion_matrix(y_test,y_pred)

# Display the confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm2)
disp.plot(cmap=plt.cm.Blues)
plt.show()
```

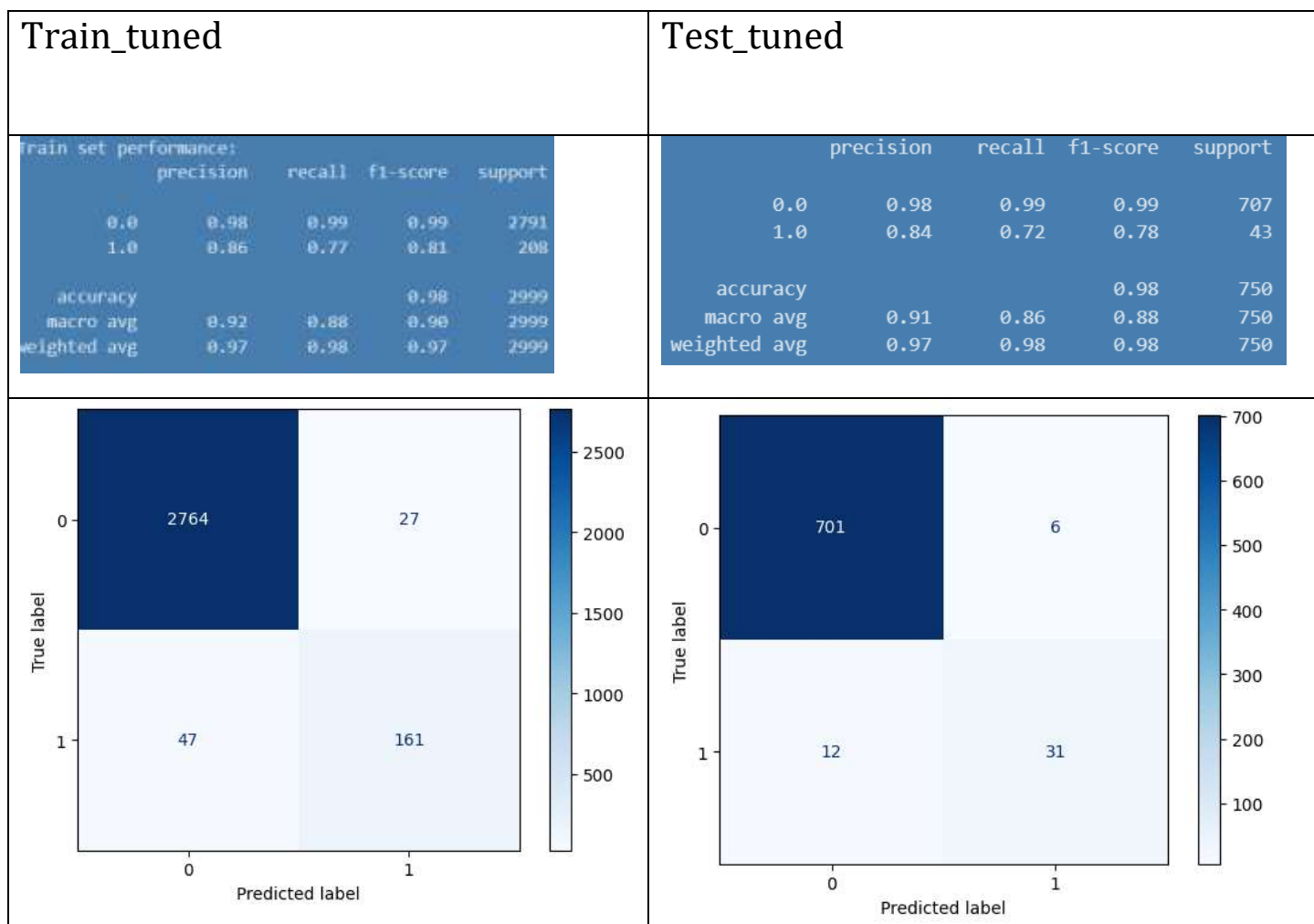
a) SGD Classifier:

The SGD Classifier updates its model with each data point, making it efficient for handling large datasets and enabling real-time learning. It uses stochastic gradient descent to optimize the model by adjusting weights based on each training example's loss. The model uses regularization to limit the size of the weights, which helps it generalize better to new, unseen data.



Observations:

The above model is the one with default values. Upon observation the model is struggling to identify the false positives. The cross validation accuracy is 97%.



Observations:

Hyper-tuning and using Grid Search, the model is certainly performing better. Following are the best parameters returned based on accuracy. The cross-validation accuracy is 97%.

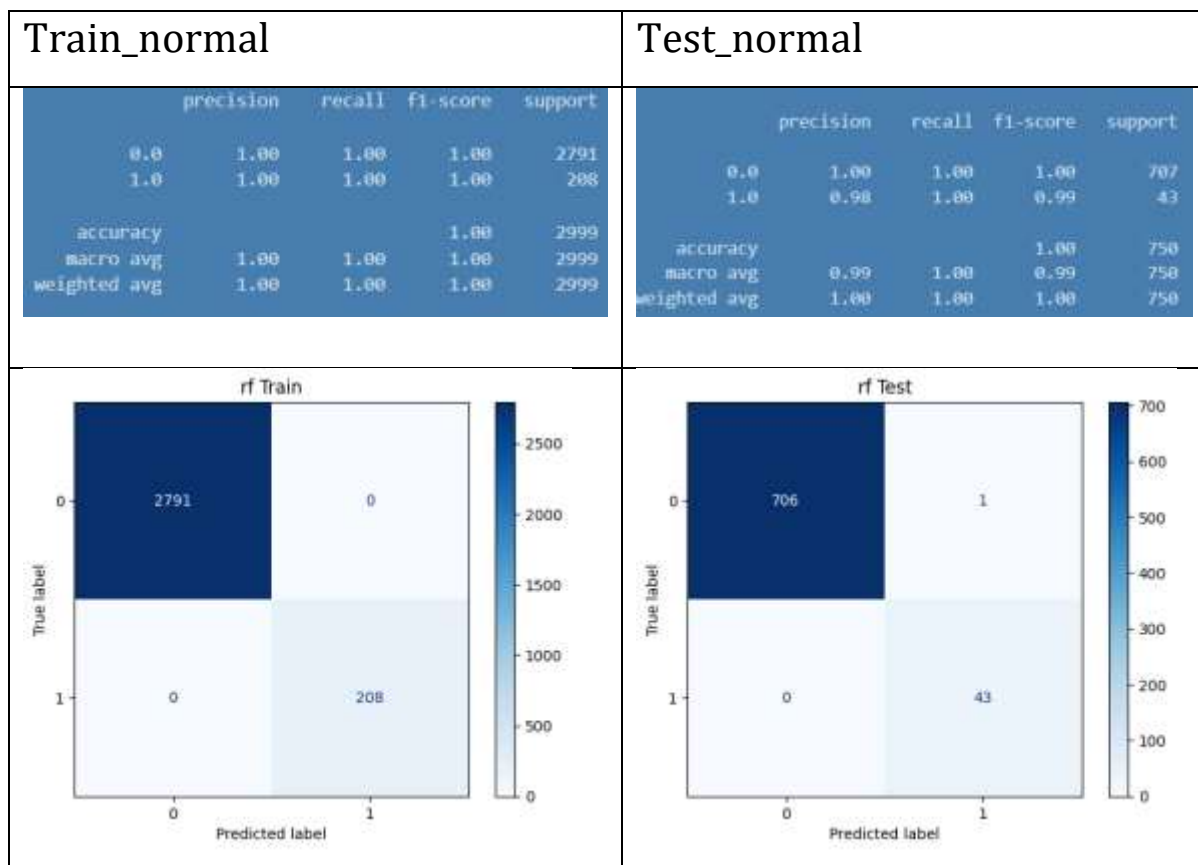
Best Parameters:

```

['sgd_alpha': 0.001,    'sgd_eta0': 0.01,    'sgd_learning_rate': 'optimal',    'sgd_loss': 'modified_huber',
'sgd_max_iter':1000, 'sgd_penalty': 'l2']
  
```

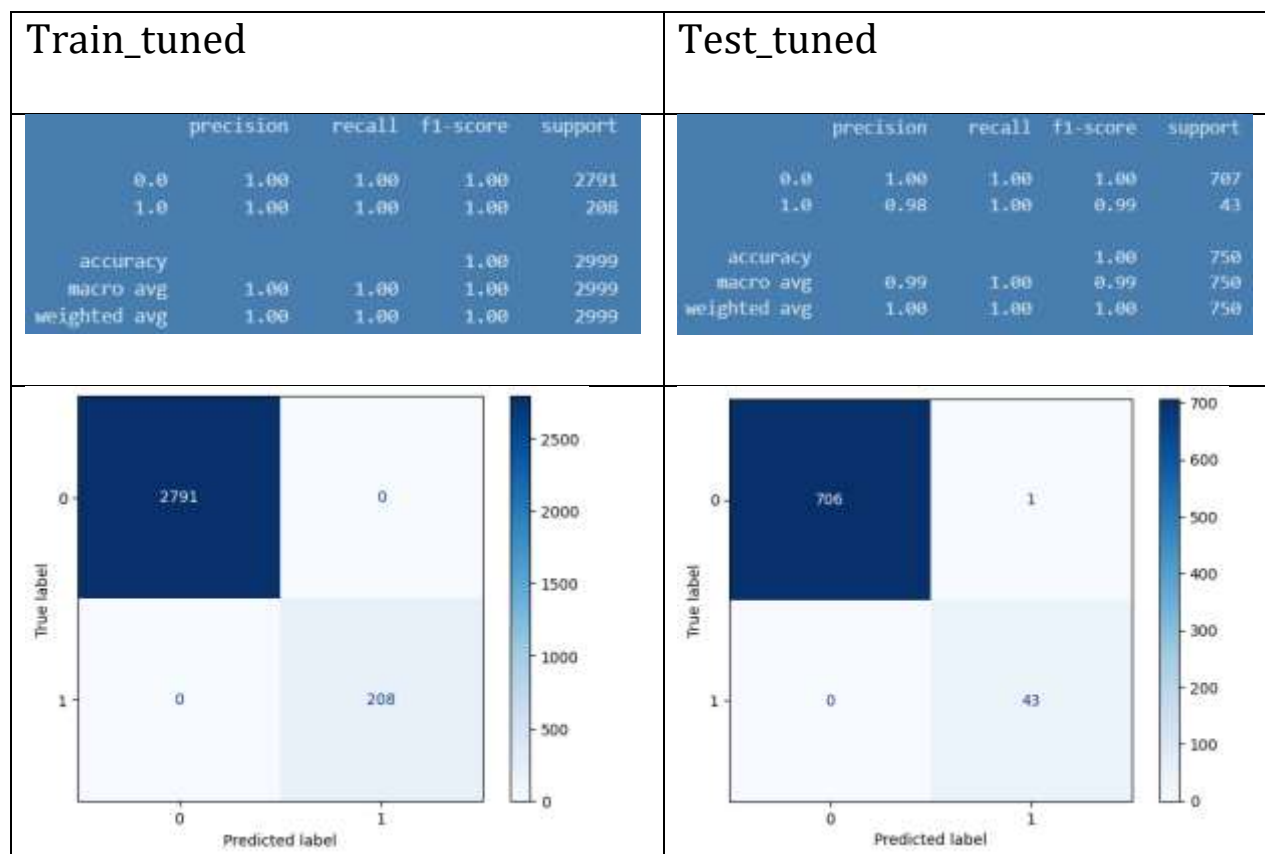
b) Random Forest Classifier:

Random Forest is an ensemble-based decision tree classifier. For classification tasks, the output of the random forest is the class selected by most trees. Random forests are a way of averaging multiple deep decision trees, trained on different parts of the same training set, with the goal of reducing the variance



Observations:

The normal model has 100% accuracy on the the train set. It also has alomost 100% accuracy on the test set. It has misclassified only a single value. The cross-validation accuracy is 99.89%



Observations:

The tuned model also has 100% accuracy on the the train set. The hypertuning has not much effect on the model. The selected parameters slightly increase the cross-validation accuracy which is 99.96%.

Best parameters:

`['rf_max_depth': None, 'rf_min_samples_leaf': 1, 'rf_min_samples_split': 2, 'rf_n_estimators':50]`

Discussion:

Different models employ various algorithms to classify the target variable, and in this case, all models are performing well. However, some models are showing signs of overfitting. Overfitting may be due to the small size of the dataset and the low correlation among the features. The key insights from the data analysis are as follows:

1. **Unbalanced Data:** The target variable is highly unbalanced, with a ratio of approximately 13:1, indicating that the dataset is heavily skewed.
 2. **Unrelated Features:** Out of the 17 features, only Tenure, Total Charges, and Monthly Charges are useful. The other features show almost zero correlation with Churn.
 3. **Model Training Challenges:** Due to the unbalanced nature of the dataset, the models tend to overfit. They accurately identify cases where churn is "No" but struggle to correctly train on cases where churn is "Yes," impaired by the small dataset size.
 4. **Ensemble Models:** Models based on ensemble methods yield similar performance metrics, indicating that the ensemble approach does not diminish the issues caused by data imbalance and overfitting.
-

Conclusion and Interpretation

The given dataset is good for practicing beginner-level machine learning due to its size and simplicity. The data is overall normally distributed with some of the features skewed.

Data pre-processing such as handling missing values, and treating outliers is crucial as it lays the foundation for running classification models. Encoding is essential for converting all categorical values to a numerical format, allowing ease in handling the dataset and model itself.

Feature engineering is another essential aspect of machine learning. It involves creating new features from existing ones that better relate to the dataset, which helps improve the model's ability to classify the target variable.

Selecting the optimal model is very important. In this case, all the classification models perform well, though some exhibit slight overfitting. Hyper-parameter tuning enhances the model's efficiency and performance.

The best-performing models are Logistic Regression, Stochastic Gradient Descent and Support vector Classifier with polynomial kernel. On the other hand, K- nearest neighbors and Random forest tend to overfit the given data.

This project has allowed to me understand the power of machine learning in solving complex problems.